CORE–3: PROGRAMMING USING C++ (UNIT – 1)

Principles of Object-Oriented Programming: The four major principles of object-oriented programming are **encapsulation, abstraction, inheritance, and polymorphism**. Some examples of object-oriented programming languages are C++, Java, C# etc.

Object-Oriented Programming (OOP) Paradigm: The important features of object–oriented programming are −

- Emphasis is on data rather than procedure.
- Programs are divided into entities known as objects.
- Data Structures are designed such that they characterize objects.
- Functions that operate on data of an object are tied together in data structures.
- Data is hidden and cannot be accessed by external functions.
- Objects communicate with each other through functions.
- New data and functions can be easily added whenever necessary.
- Follows bottom up design in program design.

Basic Concepts of OOP:

- Objects
- Classes
- Data abstraction and encapsulation
- Inheritance
- Polymorphism
- Dynamic binding
- Message passing

Objects – The instances of a class which are used in real functionality.

Classes – Basic building blocks OOP and a single entity which has data and operations on data together.

Abstraction – Specifying what to do but not how to do; a flexible feature for having a overall view of an object's functionality.

Encapsulation – Binding data and operations of data together in a single unit – A class adhere this feature

Inheritance – It is one of the property of OOPS by virtue of which object of one class can acquire or obtain property from other class.

Polymorphism – Multiple definitions for a single name - functions with same name with different functionality; saves time in investing many function names Operator and Function overloading

Dynamic binding – Binding refers to the linking of a procedure call to the code to be executed in response to the call. Dynamic binding (also known as late binding) means that the code associated with a given procedure call is not known until the time of the call at run time.

Message passing – Objects communicate through invoking methods and sending data to them. This feature of sending and receiving information among objects through function parameters is known as Message Passing.

Benefits of OOP:

- Through inheritance, we can eliminate redundant code and extend the use of existing classes.
- We can built programs from standard working modules that communicate with one another rather than, having to start writing the code from scratch. This leads to saving of development time and higher productivity.
- The principle of data hiding helps the programmers to built secure program that can't be invaded by code in other parts of the program.
- It is possible to have multiple objects to coexist without any interference.
- It is possible to map objects in the problem domain to those objects in the program.
- It is easy to partition the work in a project based on objects.
- The data-centered design approach enables us to capture more details of the model in an implementable form.
- Object-oriented systems can be easily upgraded from small to large system
- Message passing technique for communication between objects makes the interface descriptions with external system much simpler.
- Software complexity can be easily managed.

Characteristics of OOPS:

Encapsulation – It refers to the bundling of data with the methods that operate on that data.

Inheritance- This is the process by which a class can be derived from a base class with all features of base class and some of its own. This increases code reusability.

Polymorphism- This is the ability to exist in various forms. For example an operator can be overloaded so as to add two integer numbers and two floats.

Abstraction- The ability to represent data at a very conceptual level without any details is known as abstraction.

Object Oriented Languages: The languages should support several OOP concepts to claim that they are object-oriented. Depending upon the features they support, they can be classified into the following two categories:

1. Object-based programming languages
2. Object-oriented programming languages

Major features that are required for object-based programming are:

- Data encapsulation
- Data hiding and access mechanism
- Automatic initialization and clear-up objects
- Operator over loading

Object-based languages do not support inheritance and dynamic binding. Ada is a typical object-based programming language.

Object-oriented programming incorporates all of object-based programming features along with two additional features, namely, inheritance and dynamic binding. Thus,

Object-oriented programming = Object-based programming + Inheritance + dynamic binding.

Applications of OOP:

- Real Time Systems
- Simulation and Modelling
- Object-oriented databases
- Hypertext, hypermedia and expertext
- AI and Expert System
- Neural Networks and parallel programming
- Decision support and office automation systems
- CIM / CAM / CAD systems

Introduction to C++: C++ is a general purpose, case-sensitive, free-form programming language that supports object-oriented, procedural and generic programming. It was developed in 1980 by Bjarne Stroustrup at bell laboratories of AT&T (American Telephone & Telegraph), located in U.S.A. **Bjarne Stroustrup** is known as the **founder of C++ language.** It was developed for adding a feature of OOP (Object Oriented Programming) in C without significantly changing the C component. C++ programming is "relative" (called a superset) of C, it means any valid C program is also a valid C++ program.

Difference between C & C++:

C	C++
C follows the **procedural style programming.**	C++ is multi-paradigm. It supports both **procedural and object oriented.**
Data is less secured in C.	In C++, we can use modifiers for class members to make it inaccessible for outside users.
C follows the **top-down approach.**	C++ follows the **bottom-up approach.**
C does not support function overloading.	C++ supports function overloading.
In C, you can't use functions in structure.	In C++, you can use functions in structure.
C does not support reference variables.	C++ supports reference variables.
In C, **scanf() and printf()** are mainly used for input/output.	C++ mainly uses stream **cin and cout** to perform input and output operations.
Operator overloading is not possible in C.	Operator overloading is possible in C++.

C programs are divided into **procedures and modules**	C++ programs are divided into **functions and classes.**
C does not provide the feature of namespace.	C++ supports the feature of namespace.
Exception handling is not easy in C. It has to perform using other functions.	C++ provides exception handling using Try and Catch block.

Tokens: The smallest individual units in a program are known as tokens. C++ has the following tokens:

- Keyword
- Identifiers
- Punctuators
- Operators
- Constants/Literals

1. **Keywords:** Keywords are a set of words which are used for special purposes while writing a program. They are also known as reserved words. Their meaning is predefined. Few examples are: for, while, switch, break, goto, and etc.
2. **Identifiers:** An identifier is a name given to a data item in any program. The value of this identifier can change which is why it can also be called a variable.
   For example: int num=3; Here 'num' is an identifier that stores a value 3.

Rules for naming identifiers:

- They can be any combination of letters, digits and underscores. No other special character can be used.
- The name cannot start with a digit.
- C++ is a case-sensitive language. So upper and lower case characters are different.
- No keyword can be used as an identifier.

Examples :

- Valid Identifiers : num, NUM, _inPut, my_name etc
- Invalid Identifiers : 1_num (begins with a digit) , break(keyword) , my.name(uses special character)

3. **Constants/Literals:** As the name suggests, literals are those data items whose value cannot be changed during the program execution. They are four types-
   - Integer Literals
   - Floating literals
   - Character Literals
   - String Literals

Integer Literals: These are integers and they do not have any decimal/fractional part. They are of 3 types:

a) Decimal(Base 10) : Sequence of digits that does not begins with 0(zero). Eg : 798, -32, 100 etc.

b) Octal (Base 8): Sequence of digits that begin with 0. Eg: 012,056
c) Hexadecimal (Base 16): Sequence of numbers beginning with 0x/0X. Eg: 0XA

Floating Literals: These are numbers that have a decimal part. They are also called real constants.They can be signed and unsigned. For eg : 127.32, -12.9 etc

Character Literals: A single character enclosed in single quotes constitutes a character literal. For example : 'g' , 'z' etc. We can also represent non-graphic characters (can't be typed using a keyboard). This is done by using escape sequences. An escape sequence consists of a backslash followed by a single character. For example: \n is an escape sequence for newline.

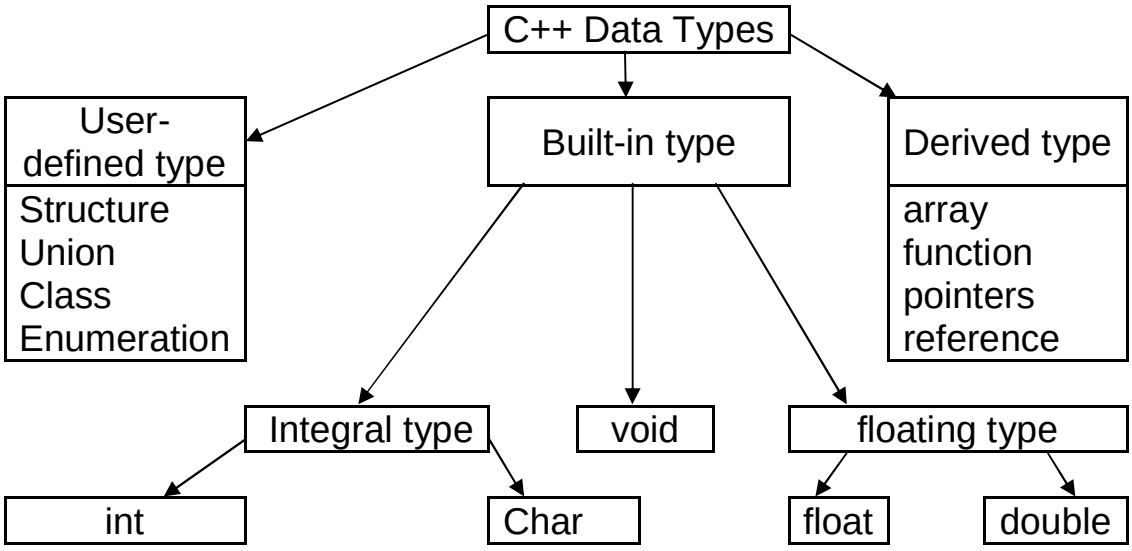
String literals: A string is a group of characters. When a string is enclosed in double quotes, it becomes a string literal. For example: "hello" is a string literal. Strings in C++ are automatically appended with a '\0' character to signify its end. So in the above example, a size of "hello" is 6.

4. Punctuators: They are mostly used for formatting statements and expressions.
Examples are [ ] { } ( ) , ; : * = etc

5. Operators: They are used for computations on data items. They are of following types:

- Arithmetic Operators : +, -, *, /
- Increment/Decrement Operators : ++, –-
- Relational Operators: ==, <, >, <=, >=, !=
- Logical operators : &&, ||, !
- Conditional Operators : ?:

Data types: Data types in C++ can be classified under various categories as shown below:



Operator in C++: All C operators are valid in C++. In addition, C++ introduces some new operators. These operators are:

::	Scope resolution operator
::*	Pointer-to-member declarator
->*	Pointer-to-member operator
.*	Pointer-to-member operator

new	Memory allocation operator
delete	Memory release operator
endl	Line feed operator
setw	Field width operator

In addition, C++ allows us to provide new definitions to some of the built in operators. That is, we can give several meanings to an operator, depending on the type of arguments used. This process is known as operator overloading.

Scope resolution operator: The :: (scope resolution) operator is used to get hidden names due to variable scopes so that we can still use them. The scope resolution operator can be used as both unary and binary. We can use the unary scope operator if a namespace scope or global scope name is hidden by a particular declaration of an equivalent name during a block or class. For example, if there is a global variable of name my_var and a local variable of name my_var, to access global my_var, you'll need to use the scope resolution operator. For example,

```
#include  <iostream.h>
int my_var = 0;
int main(void) {
        int my_var = 0;
        ::my_var = 1;  // set global my_var to 1
        my_var = 2;    // set local my_var to 2
        cout << ::my_var << ", " << my_var;
        return 0;
}
```

Output: 1, 2

The declaration of my_var declared in the main function hides the integer named my_var declared in global namespace scope. The statement ::my_var = 1 accesses the variable named my_var declared in global namespace scope.

Member Dereferencing Operators:

C++ provides a set of three pointer-to-member operators. These are-

Operator	Function
::*	To declare a pointer to a member of a class
.*	To access a member using object name and a pointer to that member
->*	To access a member using pointer to the object and a pointer to that member

Memory Management Operators:

C++ defines **new** and **delete** operators for memory allocation and deallocation respectively. These are also known as **free stroe** operators. The new operator requests for the memory allocation in heap. If the sufficient memory is available, it initializes the memory to the pointer variable and returns its address.

Syntax of new operator:	pointer_variable = new datatype;
Syntax to initialize the memory:	pointer_variable = new datatype(value);
Syntax to allocate a block of memory:	pointer_variable = new datatype[size];

Example

```
#include<iostream>
using namespace std;
int main () {
   int *ptr1  = NULL;
   ptr1 = new int;
   float *ptr2 = new float(223.324);
   int *ptr3 = new int[28];
   *ptr1 = 28;
   cout << "Value of pointer variable 1 : " <<
*ptr1 << endl;
   cout << "Value of pointer variable 2 : " <<
*ptr2 << endl;
   if (!ptr3)
      cout << "Allocation of memory failed\n";
   else {
      for (int i = 10; i < 15; i++)
         ptr3[i] = i+1;
      cout << "Value of store in block of memory: ";
      for (int i = 10; i < 15; i++)
         cout << ptr3[i] << " ";
   }
   return 0;
}
```

Output:
Value of pointer variable 1 : 28
Value of pointer variable 2 : 223.324
Value of store in block of memory: 11 12 13 14 15

The delete operator: The delete operator is used to deallocate the memory. User has privilege to deallocate the created pointer variable by this delete operator.

Syntax of delete operator: delete pointer_variable;

Syntax to delete the block of allocated memory: delete[ ] pointer_variable;

Example:

```
#include<iostream>
using namespace std;
int main () {
   int *ptr1  = NULL;
   ptr1 = new int;
   float *ptr2 = new float(299.121);
   int *ptr3 = new int[28];
   *ptr1 = 28;
   cout << "Value of pointer variable 1 : " <<
*ptr1 << endl;
   cout << "Value of pointer variable 2 : " <<
*ptr2 << endl;
   if (!ptr3)
      cout << "Allocation of memory failed\n";
else {
      for (int i = 10; i < 15; i++)
         ptr3[i] = i+1;
      cout << "Value of store in block of memory: ";
      for (int i = 10; i < 15; i++)
         cout << ptr3[i] << " ";
   }
   delete ptr1;
   delete ptr2;
   delete[] ptr3;
   return 0;
}
```

Output:
Value of pointer variable 1 : 28
Value of pointer variable 2 : 299.121
Value of store in block of memory: 11 12 13 14 15

Advantages of new operator over the function **malloc()**

- It automatically computes the size of the data object. We need not use the operator **sizeof**.
- It automatically returns the correct pointer type, so there is no need to use a type cast.
- It is possible to initialize the object while creating the memory space.
- Like any other operator, **new** and **delete** can be overloaded.

Manipulators: These are the operators that are used to format the data display. The most commonly used manipulators are **endl** and **setw**.

```
#include<iostream>
#include <iomanip>
using namespace std;
int main()        {
   int Basic = 950, Allowance = 95, Total = 1045;
        cout << setw(10) << “Basic” << setw(10) << Basic << endl;
        cout << setw(10) << “Allowance” << setw(10) << Allowance << endl;
        cout << setw(10) << “Total” << setw(10) << Total << endl;
        return 0;  }
```

Output:

```
Basic          950
Allowance       95
Total         1045
```

The type cast operator:

(type-name) expression // C notation
type-name (expression) // C++ notation

Structure of C++ Program:

C++ program structure is divided into mainly four sections, namely, headers, class definition, member functions definitions and main function.

Include files
Class declaration
Member function definition
Main function program

C++ statements:

Statements are fragments of the C++ program that are executed in sequence. The body of any function is a sequence of statements.

Example

```
int main()  {
   int n = 1;          // declaration statement
   n = n + 1;          // expression statement
   cout << "n = " << n << '\n';    // expression
statement
   return 0;              // return statement
}
```

C++ includes the following types of statements:

1) Expression statements;
2) Compound statements or blocks; (brace-enclosed sequences of statements)
3) Selection statements; (if—else, switch)
4) Iteration statements; (for, while, do-while)
5) Jump statements; (break, continue, return, goto)
6) Declaration statements; (introduce one or more identifiers into a block)
7) Try blocks; (provide the ability to catch exceptions thrown when executing other statements)
8) Atomic and synchronized blocks

Expressions and Control Structures:

Expressions and their types: An expression is a combination of constants, variables and operators arranged as per rule of the language that represents a computation. It may also include function calls which return values. Depending upon the type of operands involved in an expression or the result obtained after evaluating expression, there are different categories of an expression.

1	Constant expressions	2	Integral expressions	3	Float expressions
4	Logical expressions	5	Bitwise expressions	6	Pointer expressions
7	Relational or Boolean expressions				

Constant expressions: The expressions that comprise only constant values are called constant expressions. For examples - 20, ‘ a‘ and 2/5+30 .

Integral expressions: The expressions that produce an integer value as output after performing all types of conversions are called integral expressions. For example,

x, 6*x-y and 10 + int(5.0) are integral expressions. Here, x and yare variables of type int.

Float expressions: The expressions that produce floating-point value as output after performing all types of conversions are called float expressions. For example, 9.25, x-y and 9+ float (7) are float expressions. Here, x and y are variables of type float.

Logical expressions: The expressions that produce a bool type value after combining two or more relational expressions are called logical expressions. For example, x==5 && m==5 and y>x || m<=n are logical expressions.

Bitwise expressions: The expressions which manipulate data at bit level are called bitwise expressions. For example, a >> 4 and b<< 2 are bitwise expressions.

Pointer expressions: The expressions that give address values as output are called pointer expressions. For example, &x, ptr and ptr+1 are pointer expressions. Here, x is a variable of any type and ptr is a pointer.

Relational or Boolean expressions: The expressions that produce a bool type value, that is, either true or false are called relational or Boolean expressions. For example, x + y<100, m + n==a-b and a>=b + c are relational expressions.

Special assignment expressions: An expression can be categorized further depending upon the way the values are assigned to the variables.

Chained assignment: Chained assignment is an assignment expression in which the same value is assigned to more than one variable, using a single statement. For example,

a = (b=20); or a=b=20;

In these statements, value 20 is assigned to variable b and then to variable a. Note that variables cannot be initialized at the time of declaration using chained assignment. For example,

```
int a=b=30; // illegal
int a=30, int b=30; //valid
```

Embedded assignment: Embedded assignment is an assignment expression, which is enclosed within another assignment expression.

For example, a=20+(b=30); //equivalent to b=30; a=20+30;

In this statement, the value 30 is assigned to variable b and then the result of (20+ 30) that is, 50 is assigned to variable a. Note that the expression (b=30) is an embedded assignment.

Compound Assignment: Compound Assignment is an assignment expression, which uses a compound assignment operator that is a combination of the

assignment operator with a binary arithmetic operator. For example, consider this statement.

a + =20; //equivalent to a=a+20;

In this statement, the operator += is a compound assignment operator, also known as short-hand assignment operator.

Type Conversion: When an expression consists of mixed data types then they are converted to the same type while evaluation, to avoid compatibility issues. This is accomplished by type conversion, which is defined as the process of converting one predefined data type into another. Type conversions are of two types, namely, *implicit conversions* and *explicit conversions* also known as ***typecasting****.*

Implicit Conversions: Implicit conversion, also known as automatic type conversion refers to the type conversion that is automatically performed by the compiler. Whenever compiler confronts a mixed type expression, first of all char and short int values are converted to int. This conversion is known as integral promotion. After applying this conversion, all the other operands are converted to the type of the largest operand and the result is of the type of the largest operand. The Table shown here illustrates the implicit conversion of data type starting from the smallest to largest data type. For example, in expression 5 + 4.25, the compiler converts the int into float as float is larger than int and then performs the addition.

Order of Data Types

Data Types	
chat short int	**Smallest**
int	
unsigned	
long int	
unsigned int	
float	
double	
long double	**Largest**

Typecasting: Typecasting refers to the type conversion that is performed explicitly using type cast operator. In C++, typecasting can be performed by using two different forms as follows.

data_type (expression) //expression in parentheses

(data_type)expression //data type in parentheses

For example, float (num)+ 3.5; //num is of int type

In this example, float () acts as a conversion function which converts int to float. However, this form of conversion cannot be used in some situations. For example,

ptr=int * (x) *;*

In such cases, conversion can be done using the second form of typecasting (which is basically C-style typecasting) i.e. ptr=(int*)x;

Control Structures: There are three types of control structures available in C++

1. Sequence structure (straight line paths)
2. Selection structure (one or many branches): implemented using if, if else and switch statements
3. Loop structure (repetition of a set of activities): implemented using **while, do while and for statements.**

The if statement is implemented in two forms

- Simple if statement
- If... else setatement

The switch statement is a multiple-branching statement where, based on a condition, the control is transferred to one of the many possible points. These are as follows:

```
From 1
If(expression is true)
{
   Action 1;
}
Action 2;
Action 3;
```

```
From 2
if(expression is true){
   Action 1;
}
else{
   Action 2;
}
Action 3;
```

```
switch(expression)
{
    Case 1:  Action 1;
    Case 2:  Action 2;
    Case 3:  Action 3;
    default:  Action 4;
}
Action 5;
```

The do-while, while and for statement: The do-while statement is an exit-controlled loop. Based on a condition, the control is transferred back to a particular point in the program. The while statement is an entry-controlled loop. The for statement is an entry-controlled loop and is used when an action is to be repeated for a predetermined number of items. These are as follows:

do-while

```
do
{
   Action1;
}while(condition is true);
Action 2;
```

while

```
while(condition is true)
{
   Action1;
}
Action 2;
```

for

```
for(initial value; test; increment)
{
   Action1;
}
Action 2;
```

Difference between while and do-while loop:

3	The iterations do not occur if, the condition at the first iteration appears false.	The iteration occurs at least once even if the condition is false at the first iteration.
4	This is an entry-controlled loop	This is an exit-controlled loop
5	Semicolon is not used	Semicolon is used at the end of the loop

Functions in C++: A function is a group of statements that together perform a task. Every C++ program has at least one function, which is main(). The function in C++ language is also known as procedure or subroutine in other programming languages.

Example:

```
Void show();          // function declaration
Main()
{  .............;
   Show();            // function call
   .............;  }
Void show(){        // function definition
   .............;     // function body
   .............;  }
```

Advantage of functions:

- **Code Reusability:** By creating functions in C++, you can call it many times. So we don't need to write the same code again and again.
- **Code optimization:** It makes the code optimized; we don't need to write much code.

Types of Functions:

1. **Library Functions:** are the functions which are declared in the C++ header files such as ceil(x), cos(x), exp(x), etc.
2. **User-defined functions:** are the functions which are created by the programmer, It reduces complexity of a big program and optimizes the code.

Function prototyping: A function prototype or function interface is a declaration of a function that specifies the function's name, return type, number of arguments passed to the function, data types of the each of the arguments and the order in which the arguments are passed to the function but omits the function body. Prototype of a function is also called **signature of the function**.

Example: float volume(int x, float y, float z);

Call by value and call by reference in C++: There are three ways to pass value or data to function in C language: call by value, call by reference and call by pointer. Original value is not modified in call by value but it is modified in call by reference. Call by pointer copies the address of an argument into the formal parameter. Inside the function, the address is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.

Example:

Call by value

```
#include<iostream>
using namespace std;
void change(int data);
int main()  {
    int data = 3;
    change(data);
   cout << "Value of the data is: " << data<< endl;
  return 0;
}
void change(int data)  {
    data = 5;
}
```

Output:

```
Value of the data is: 3
```

Call by reference

```
#include<iostream>
using namespace std;
void swap(int &a, int &b)  {
 int t;  t=a;   a=b;   b=t;
}
int main()   {
 int x=500, y=100;
 swap(x, y);  // passing reference to function
 cout<<"Value of x is: "<<x<<endl;
 cout<<"Value of y is: "<<y<<endl;
 return 0;
}
```

Output:

```
Value of x is: 100
Value of y is: 500
```

Difference between call by value and call by reference in C++

No.	Call by value	Call by reference
1	A copy of value is passed to the function	An address of value is passed to the function
2	Changes made inside the function is not reflected on other functions	Changes made inside the function is reflected outside the function also
3	Actual and formal arguments will be created in different memory location	Actual and formal arguments will be created in same memory location

Inline Functions: The inline functions are a C++ enhancement feature to reduce the execution time of a program. Functions can be instructed to compiler to make them inline so that compiler can replace those function definition wherever those are being called. Compiler replaces the definition of inline functions at compile time instead of referring function definition at runtime. Inlining is only a request to the compiler, not a command. Compiler can ignore the request for inlining. Compiler may not perform inlining in such circumstances like:
1) If a function contains a loop. (for, while, do-while)
2) If a function contains static variables.
3) If a function is recursive.
4) If a function return type is other than void, and the return statement doesn't exist in function body.
5) If a function contains switch or goto statement.

Why to use: In many places we create the functions for small work/functionality which contain simple and less number of executable instruction. Imagine their calling overhead each time they are being called by callers. When a normal function call instruction is encountered, the program stores the memory address of the instructions immediately following the function call statement, loads the function being called into the memory, copies argument values, jumps to the memory location of the called function, executes the function codes, stores the return value of the function, and then jumps back to the address of the instruction that was saved just before executing the called function. This makes too much run time overhead.

The C++ inline function provides an alternative. With inline keyword, the compiler replaces the function call statement with the function code itself (process called expansion) and then compiles the entire code. Thus, with inline functions, the compiler does not have to jump to another location to execute the function, and then jump back as the code of the called function is already available to the calling program.

How to make function inline: To make any function as inline, start its definitions with the keyword "inline". If a member function is defined inside a class it becomes inline by default.

Example:

```
inline int cube(int s){
    return s*s*s;
}
int main()
{
    cout << cube(3);
    return 0;
}
//Output: 27
```

```
class A
{
 Public:
    inline int cube(int s)
    {
       return (s*s*s);
    }
};
```

```
class A
{
 Public:
    int cube(int s)
    {
       return (s*s*s);
    }
};
```

```
//Making  an outside
//Function Inline
class A{
 Public:
    int cube(int s);
};
inline int A::cube(int s){
   return (s*s*s);
}
```

Advantages of using inline function:

1. It speeds up your program by avoiding function calling overhead.
2. It save overhead of variables push/pop on the stack, when function calling happens.
3. It save overhead of return call from a function.
4. It increases locality of reference by utilizing instruction cache.
5. By marking it as inline, you can put a function definition in a header file (i.e. it can be included in multiple compilation units, without the linker complaining)

Disadvantages of using inline function:

1. It increases the executable size due to code expansion.
2. C++ inlining is resolved at compile time which means if we change the code of the inline function, we need to recompile all the code using it.
3. When used in a header, it makes your header file larger with information which users don't care.
4. As mentioned above it increases the executable size, which may cause thrashing in memory. More number of page fault bringing down your program performance.
5. Sometimes not useful for example in embedded system where large executable size is not preferred at all due to memory constraints.

When to use inline function: Function can be made as inline as per programmer need. Some useful recommendations may be

1. Use inline function when performance is needed.
2. Use inline function over macros.
3. Prefer to use inline keyword outside the class with the function definition to hide implementation details.

Key Points -

1. It's just a suggestion not compulsion. Compiler may or may not inline the functions you marked as inline. It may also decide to inline functions not marked as inline at compilation or linking time.
2. Inline works like a copy/paste controlled by the compiler, which is quite different from a pre-processor macro: The macro will be forcibly inlined, will pollute all the namespaces and code, won't be easy to debug.
3. All the member function declared and defined within class are Inline by default. So no need to define explicitly.
4. Virtual methods are not supposed to be inlinable. Still, sometimes, when the compiler can know for sure the type of the object (i.e. the object was declared and

constructed inside the same function body), even a virtual function will be inlined because the compiler knows exactly the type of the object.
5. Template methods/functions are not always inlined (their presence in a header will not make them automatically inline).

Difference between inline function and macro:

Inline function	Macro
These are functions provide by C++.	Macros are pre-processor directives.
Inline keyword is used to declare the function as inline	#define is used to declare the macro.
Inline functions are parsed by the compiler.	Macros are expanded by the C++ pre-processor.
Inline function can access the data member of the class.	Macros cannot access the data member of the class.
Compiler replaces the function call with the function code.	C++ pre-processor replaces every occurrence of macro template with its corresponding definition.
Inline function follows strict parameter type checking.	Macro does not follow parameter type checking.
Inline function may or may not be expanded by the compiler. It depends on the compiler whether to expand the function inline or not.	Macros are always expanded.

Example:
inline int cube(int s){
return s*s*s;
}
	Example:
	
	#define cube(s) (s*s*s)

Default Arguments: A default argument is a value provided in a function declaration that is automatically assigned by the compiler if the caller of the function doesn't provide a value for the argument with a default value. Consider the following function declaration:

```
int MyFunc(int a, int b, int c = 12);
```

This function takes three arguments, of which the last one has a default of twelve. The programmer may call this function in two ways:

```
int result = MyFunc(1, 2, 3);
result = MyFunc(1, 2);
```

In the first case the value for the argument called *c* is specified as normal. In the second case, the argument is omitted, and the default value of *12* will be used instead. There is no means to know if the argument has been specified by the caller or if the default value was used. The above-mentioned method is especially useful when one wants to set default criteria so that the function can be called with or without parameters. One important point to note is that only the trailing arguments can have default values. For example,

```
void add(int a, int b = 3, int c, int d = 4);      // illegal
void add(int a, int b = 3, int c, int d);          // illegal
void add(int a, int b, int c = 3, int d = 4);      // legal
```

Constant Arguments: In C++, an argument to a function can be declared as const. The argument with constant value should be initialized during the function declaration.

Syntax: type function_name(const data_type variable_name=value);
Example, int max(const int a=3, int b); //function prototype

The qualifier const tells the compiler that the value of that argument cannot be modified, and any such attempt will generate a compile time error. We can indirectly modify the value of the const argument by using pointer. Compile time error occurs only when we pass arguments by reference or pointer.

Friend function: A friend function of a class is defined outside that class' scope but it has the right to access all private and protected members of the class. Even though the prototypes for friend functions appear in the class definition, friends are not member functions. To declare a function as a friend of a class, precede the function prototype in the class definition with keyword **friend** as shown in the example.

To declare all member functions of class ClassTwo as friends of class ClassOne, place the following declaration in the definition of class ClassOne −

```
friend class ClassTwo;
```

Characteristics of a Friend function:

- The function is not in the scope of the class to which it has been declared as a **friend**.
- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- Unlike member functions, it cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- It can be declared either in the private or the public part.
- Usually, it has the objects as arguments.

Example:

```
#include<iostream>
using namespace std;
class Box {
   double width;
   public:
      friend void printWidth( Box box );
      void setWidth( double wid );
};
void Box::setWidth( double wid ) {
   width = wid;
}
void printWidth( Box box ) {
   cout << "Width of box : " << box.width
<<endl;
}
int main() {
   Box box;
      box.setWidth(10.0);
      printWidth( box );
   return 0;
}
```

Output: Width of box : 10